

DOCKER, DOCKER DESKTOP AND LINUX LAB PRACTICAL NOTES

Ubuntu and Alpine Linux containers • CLI commands • File management • Shell scripting •
Docker operations

Subject: Linux and Operating System

Format: Student-friendly practical notes

Environment: Windows with Docker Desktop, or Docker Engine on Linux

Purpose: Learn Linux command-line fundamentals using disposable, repeatable container environments without installing a separate desktop operating system.

UNIT 1: INTRODUCTION TO DOCKER

1.1 What is Docker?

Docker is an open-source platform used to build, run, manage and distribute applications in isolated environments called **containers**. A container packages an application with its required libraries, dependencies and configuration.

Example: A Windows user can run an Ubuntu container and practice Linux commands such as `ls`, `pwd`, `mkdir`, `touch`, `cp`, `mv`, `ps` and `top`.

1.2 Why use Docker?

- Run Linux environments on Windows, Linux or macOS.
- Practice Linux commands without installing a separate OS.
- Create isolated and repeatable lab environments.
- Run Ubuntu, Alpine, Debian and other Linux distributions.
- Practice file management, shell scripting, networking and processes.
- Create, start, stop and remove environments quickly.

1.3 What is Docker Desktop?

Docker Desktop is a graphical application for installing, configuring and managing Docker on supported desktop operating systems. It provides a GUI for containers and images, logs, resource monitoring, settings, and Docker Compose support.

1.4 Docker and Docker Desktop

Docker	Docker Desktop
Container platform and engine ecosystem	Desktop application for managing Docker
Can be operated using the CLI	Provides GUI and integrated tools
Used on Linux servers and desktops	Designed for supported desktop platforms
Example: <code>docker run ubuntu</code>	Manage containers and images in a dashboard

1.5 Important Docker terms

Term	Meaning
Docker Engine	Software that builds and runs containers.
Image	Read-only template used to create containers.
Container	Running or stopped instance of an image.
Docker Hub	Online registry for container images.
Dockerfile	Text instructions for building an image.
Volume	Docker-managed persistent storage.
Network	Mechanism for container communication.
Docker Compose	Tool for defining and running multi-container applications.

1.6 Virtual machine vs container

Virtual machine	Docker container
Runs a guest OS with its own kernel	Shares the host Linux kernel
Usually uses more resources	Usually lightweight
Boots a full operating system	Starts an isolated process environment
Uses a hypervisor	Uses container runtime and kernel isolation
Useful for full OS virtualization	Useful for command labs and app environments

Important: An Ubuntu container is not a full virtual machine. It provides a Linux user space and shares the host's Linux kernel. On Windows, Docker Desktop commonly runs Linux containers through a lightweight Linux environment such as WSL 2.

UNIT 2: INSTALL DOCKER DESKTOP ON WINDOWS

2.1 Basic requirements

- Supported Windows 10/11 edition and build.
- Hardware virtualization enabled in BIOS/UEFI.
- WSL 2 enabled for the recommended Linux-container setup.
- Adequate RAM; 8 GB or more is preferable for comfortable lab use.
- Internet access to download Docker and Linux images.

2.2 Installation steps

Step 1: Visit [Docker Desktop official website](#) and download Docker Desktop for Windows.

Step 2: Open PowerShell as Administrator and install WSL if needed:

```
wsl --install
wsl --status
wsl --list --verbose
```

Step 3: Run the Docker Desktop installer. Select the WSL 2 backend if offered, complete installation and restart if requested.

Step 4: Open Docker Desktop from the Start menu, complete first-run prompts, and wait until Docker Engine is running.

2.3 Verify installation

```
docker --version
docker info
docker run hello-world
```

A successful hello-world run prints a message beginning "Hello from Docker!" and confirms that Docker can launch a test container.

Official setup guide: [Docker Desktop for Windows installation](#)

UNIT 3: RUN UBUNTU LINUX IN DOCKER

3.1 About Ubuntu

Ubuntu is a Debian-based Linux distribution used for desktops, servers, development and education. An Ubuntu container provides common Linux utilities and access to APT package management.

3.2 Download and inspect the image

```
docker pull ubuntu:24.04
docker images
```

The `pull` command downloads the image; `images` lists locally available images.

3.3 Create an interactive Ubuntu container

```
docker run -it --name ubuntu-lab ubuntu:24.04
```

Option	Meaning
<code>docker run</code>	Create and start a container.
<code>-i</code>	Keep standard input open.
<code>-t</code>	Allocate a terminal.
<code>--name</code>	Assign a container name.
<code>ubuntu:24.04</code>	Image and release tag.

The shell prompt may look like `root@container-id:/#`. Commands below run inside the container.

3.4 Basic Linux commands

```
pwd
uname -a
cat /etc/os-release
whoami
date
ls
ls -lah
mkdir linux-lab
cd linux-lab
touch file1.txt file2.txt
echo "Welcome to Ubuntu Linux Lab" > file1.txt
cat file1.txt
cp file1.txt backup.txt
mv backup.txt newfile.txt
rm newfile.txt
cd /
```

3.5 Install useful Ubuntu utilities

```
apt update
apt install -y nano vim curl wget iproute2 iputils-ping \
  procps net-tools dnsutils tree htop sudo
nano --version
curl --version
ip -V
tree --version
```

A minimal Ubuntu image does not contain every utility. Update the package index before installing packages.

3.6 File and directory practice

```
mkdir -p /home/student/linux-lab
cd /home/student/linux-lab
mkdir documents scripts backup
touch documents/notes.txt scripts/test.sh
tree /home/student/linux-lab
cp documents/notes.txt backup/
mv backup/notes.txt backup/old-notes.txt
chmod 755 scripts/test.sh
ls -l scripts/
```

3.7 Shell scripting

Create a file named `hello.sh` using Nano:

```
nano hello.sh
```

Enter and save the following script. In Nano, use Ctrl+O, Enter, then Ctrl+X.

```
#!/bin/bash
echo "Welcome to Ubuntu Linux Lab"
echo "Current User: $(whoami)"
echo "Current Directory: $(pwd)"
echo "Today's Date: $(date)"

chmod +x hello.sh
./hello.sh
```

3.8 Exit, restart and re-enter

```
exit
docker ps -a
docker start -ai ubuntu-lab
# Or start in background, then open a shell:
docker start ubuntu-lab
docker exec -it ubuntu-lab bash
```

Stopping a container does not remove it. Its writable layer remains while the container exists. Removing the container deletes that layer; use a volume or bind mount for data that must survive container removal.

UNIT 4: RUN ALPINE LINUX IN DOCKER

4.1 About Alpine Linux

Alpine Linux is a small, security-oriented Linux distribution. It is useful for learning minimal Linux environments, BusyBox utilities, the ash shell and APK package management.

4.2 Download and run Alpine

```
docker pull alpine:latest
docker images
docker run -it --name alpine-lab alpine:latest
```

The default shell is commonly BusyBox ash, and the prompt may appear as / #.

4.3 Basic Alpine commands

```
pwd
ls
uname -a
cat /etc/alpine-release
whoami
date
mkdir linux-practice
cd linux-practice
touch notes.txt
echo "Welcome to Alpine Linux" > notes.txt
cat notes.txt
ls -la
```

4.4 Install Alpine packages

```
apk update
apk add bash nano vim curl wget iproute2 iputils \
  procs bind-tools tree httpd
bash --version
nano --version
curl --version
ip -V
```

Alpine uses apk, not APT. Common commands: `apk add package` to install, `apk search term` to search, and `apk del package` to remove.

4.5 Shell scripting in Alpine

Install Bash if a practical requires Bash-specific syntax; otherwise, simple scripts can run with Alpine's default ash.

```
apk add bash
nano hello.sh
```

```
#!/bin/bash
echo "Welcome to Alpine Linux"
echo "Release:"
cat /etc/alpine-release
echo "User: $(whoami)"
echo "Directory: $(pwd)"

chmod +x hello.sh
./hello.sh
```

4.6 Ubuntu and Alpine comparison

Feature	Ubuntu 24.04	Alpine Linux
Base	Debian-based	Independent, musl-based
Typical shell	Bash available	BusyBox ash
Package manager	APT	APK
Install command	apt install	apk add
C library	glibc	musl
Image size	Generally larger	Generally smaller
Common use	General Linux and development	Minimal containers and lightweight environments

Lab guidance: Ubuntu is useful for broad introductory Linux practice; Alpine is useful for minimal environments and alternate package/shell tools. Neither is a full VM. Some administration exercises require a VM or specially configured container.

UNIT 5: DOCKER CONTAINER MANAGEMENT

5.1 List containers

```
docker ps
docker ps -a
```

docker ps lists running containers; docker ps -a includes stopped containers.

5.2 Start, stop, restart and enter

```
docker start ubuntu-lab
docker stop ubuntu-lab
docker restart ubuntu-lab
docker exec -it ubuntu-lab bash
docker exec -it alpine-lab sh
```

5.3 Run containers in detached mode

```
docker run -dit --name ubuntu-background ubuntu:24.04 bash
docker run -dit --name alpine-background alpine:latest sh
docker ps
docker exec -it ubuntu-background bash
docker exec -it alpine-background sh
```

Detached mode runs the container in the background. Use docker exec to open another shell in a running container.

5.4 Logs, processes and resource use

```
docker logs ubuntu-background
docker logs -f ubuntu-background
docker stats
docker inspect ubuntu-lab
docker top ubuntu-lab
```

5.5 Remove containers and images

```
docker stop ubuntu-lab
docker rm ubuntu-lab
docker rmi ubuntu:24.04
docker container prune
docker image prune
docker system df
```

Caution: Prune commands remove unused resources. Inspect containers, images and volumes before cleanup so that lab work is not lost.

UNIT 6: SHARE FILES BETWEEN WINDOWS AND LINUX

6.1 Bind mounts and volumes

A **bind mount** shares a chosen host folder with a container. A **volume** is Docker-managed storage that persists independently of a container.

6.2 Bind mount a Windows folder

In Windows PowerShell, create a folder:

```
mkdir C:\LinuxLab
```

Run Ubuntu with that folder mounted at /lab:

```
docker run -it --name ubuntu-shared \
  --mount type=bind,source=C:\LinuxLab,target=/lab \
  ubuntu:24.04 bash
```

Inside Ubuntu:

```
cd /lab
pwd
touch practical1.txt
echo "Linux practical exercise" > practical1.txt
cat practical1.txt
exit
```

Open C:\LinuxLab in File Explorer; the file should be visible on Windows.

6.3 Create and reuse a Docker volume

```
docker volume create linux-data
docker run -it --name ubuntu-volume \
  --mount source=linux-data,target=/data \
  ubuntu:24.04 bash
```

Inside the container, create a file and exit:

```
cd /data
echo "Persistent Linux lab data" > lab.txt
cat lab.txt
exit
```

Remove the container, then create another container using the same volume:

```
docker rm ubuntu-volume
docker run -it --rm \
  --mount source=linux-data,target=/data \
  ubuntu:24.04 bash
cat /data/lab.txt
```

The file remains available because it is stored in the Docker volume.

UNIT 7: PRACTICAL LINUX LAB EXERCISES

Exercise 1: Basic Linux commands

- Display the current directory and list files.
- Check the Linux distribution and kernel version.

- Display the current user and date.

Exercise 2: File and directory management

- Create a directory named LinuxLab.
- Create five text files and display their contents.
- Copy, rename, move and delete files.

Exercise 3: User and permission concepts

- Display the current user and group information.
- Create sample files and practice chmod permissions.
- Compare read, write and execute permissions.

Exercise 4: Shell scripting

- Write a script that displays your name, date and current directory.
- Create a script that accepts two numbers and displays their sum.
- Write an if-else script to check whether a number is even or odd.

Exercise 5: Process management

- Display running processes using ps.
- Run a long-running process and identify its PID.
- Terminate the process using kill.

Exercise 6: Networking

- Display interfaces using ip addr.
- Display routes using ip route.
- Use ping and DNS lookup tools where networking is available.

Exercise 7: Package management

- Update Ubuntu package indexes using apt.
- Install and test nano, curl and tree.
- Repeat package installation and removal using Alpine apk.

Exercise 8: Docker management

- Create Ubuntu and Alpine containers.
- Stop, start and restart both containers.
- Compare docker ps, docker images and docker inspect output.

UNIT 8: DOCKER COMMAND CHEAT SHEET

Command	Purpose
<code>docker --version</code>	Check Docker CLI version
<code>docker info</code>	Show Docker Engine information
<code>docker pull ubuntu:24.04</code>	Download Ubuntu image
<code>docker pull alpine:latest</code>	Download Alpine image
<code>docker images</code>	List local images
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers
<code>docker run -it ubuntu:24.04 bash</code>	Start interactive Ubuntu
<code>docker run -it alpine:latest sh</code>	Start interactive Alpine
<code>docker start ubuntu-lab</code>	Start existing container
<code>docker stop ubuntu-lab</code>	Stop a container
<code>docker restart ubuntu-lab</code>	Restart a container
<code>docker exec -it ubuntu-lab bash</code>	Open Ubuntu shell
<code>docker exec -it alpine-lab sh</code>	Open Alpine shell
<code>docker logs ubuntu-lab</code>	Display container logs
<code>docker stats</code>	Monitor resource usage
<code>docker inspect ubuntu-lab</code>	Show container details
<code>docker rm ubuntu-lab</code>	Remove a container
<code>docker rmi ubuntu:24.04</code>	Remove an image
<code>docker volume ls</code>	List volumes
<code>docker network ls</code>	List networks
<code>docker system df</code>	Show Docker disk usage

UNIT 9: IMPORTANT QUESTIONS

Short-answer questions

- Define Docker and explain its purpose.
- What is Docker Desktop?
- Differentiate between a Docker image and a container.
- What is Docker Hub?
- Differentiate Ubuntu and Alpine Linux.
- Explain `docker pull`, `docker run` and `docker exec`.
- What is a Docker volume?
- Differentiate a Docker container and a virtual machine.

Long-answer and practical questions

- Explain Docker architecture with a suitable diagram.
- Describe Docker Desktop installation and configuration on Windows.
- Demonstrate creating and managing an Ubuntu container.
- Install Alpine Linux using Docker and demonstrate basic commands.
- Compare Ubuntu and Alpine in package management, shell, size and use cases.
- Explain the Docker container lifecycle with commands.
- Demonstrate Windows-to-Ubuntu file sharing using a bind mount.
- Explain Docker volumes and demonstrate data persistence after container removal.

SUGGESTED LAB SETUP AND SAFETY NOTES

- Install Docker Desktop on Windows and verify it with hello-world.
- Download Ubuntu and Alpine images; create separate named containers.
- Practice Linux CLI, file management, scripting, processes and networking.
- Use bind mounts or volumes to preserve practical files.
- Stop and restart containers between sessions; avoid removing containers until work is saved.
- Use a full virtual machine for boot, systemd, kernel-module and full OS administration exercises.

Official documentation and images

- Docker Desktop for Windows: docs.docker.com/desktop/setup/install/windows-install
- Docker container concepts: [docs.docker.com container guide](https://docs.docker.com/container/)
- Official Ubuntu image: hub.docker.com/_/ubuntu
- Official Alpine image: hub.docker.com/_/alpine
- Docker storage: docs.docker.com/engine/storage